Nombre: Pedro Omar Cardona Alvarez **Nombre**: Kloudenn Jafed Jiménez Valdivieso **Nombre:** Leonardo Acosta Morales	**Matrícula**: 2226579 **Matrícula**: 2226639 **Matrícula**: 2226717
Nombre del curso: Componentes de lógica de Negocio.	**Nombre del profesor**: Miguel Ángel Gómez Marroquín
Modulo Netacad:	**Actividad**: TechStore_U4 – Equipo 14
Fecha: 26/10/2026	

Bibliografia (Formato APA)

- Fowler, M. (2003). Patterns of Enterprise Application Architecture. Addison-Wesley.
- Gómez Marroquín, M. Á. (2026). Componentes de Lógica de Negocio. Microsoft Teams. UANL FIME.
- Martin, R. C. (2017). Clean Architecture: A Craftsman's Guide to Software Structure and Design. Prentice Hall.
- Millett, S., & Tune, N. (2015). Patterns, Principles, and Practices of Domain-Driven Design. Wrox.

Objetivo:
El objetivo de esta actividad es desarrollar un programa para administrar pedidos de una tienda ficticia (TechStore), aplicando una arquitectura basada en Domain-Driven Design (DDD). Se busca transformar el análisis de negocio en una implementación real utilizando Entidades, Value Objects, validación estricta de invariantes mediante el Specification Pattern, control del ciclo de vida con una Máquina de Estados, emisión de Domain Events, y exponer las operaciones de manera segura a través de una API REST con FastAPI, asegurando su correcto funcionamiento mediante pruebas automatizadas con pytest.

Procedimiento:

1.- Repositorio GitHub

Enlace al repositorio: https://github.com/kloudenn29/TechStore-U4
Estructura entregada:

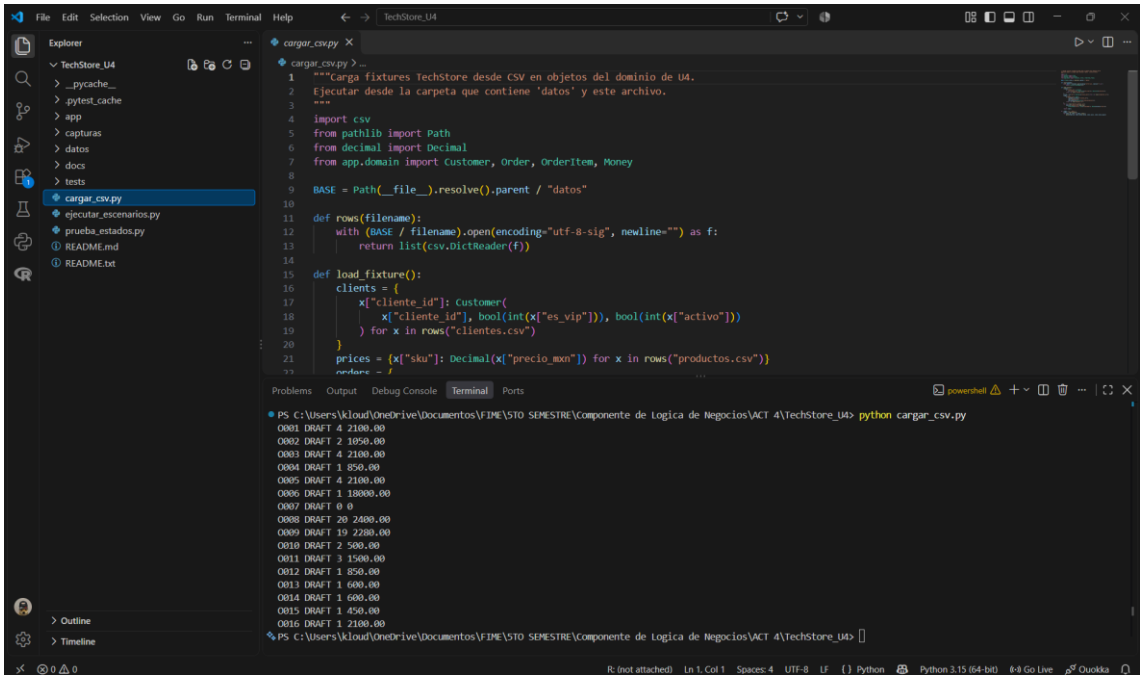
- /app (dominio, API, especificaciones)
- /tests (pruebas pytest)
- /datos (archivos CSV)
-/docs (archivos pdf como la bitacora, diagramas y este reporte adjuntado)

1.1 - README.md

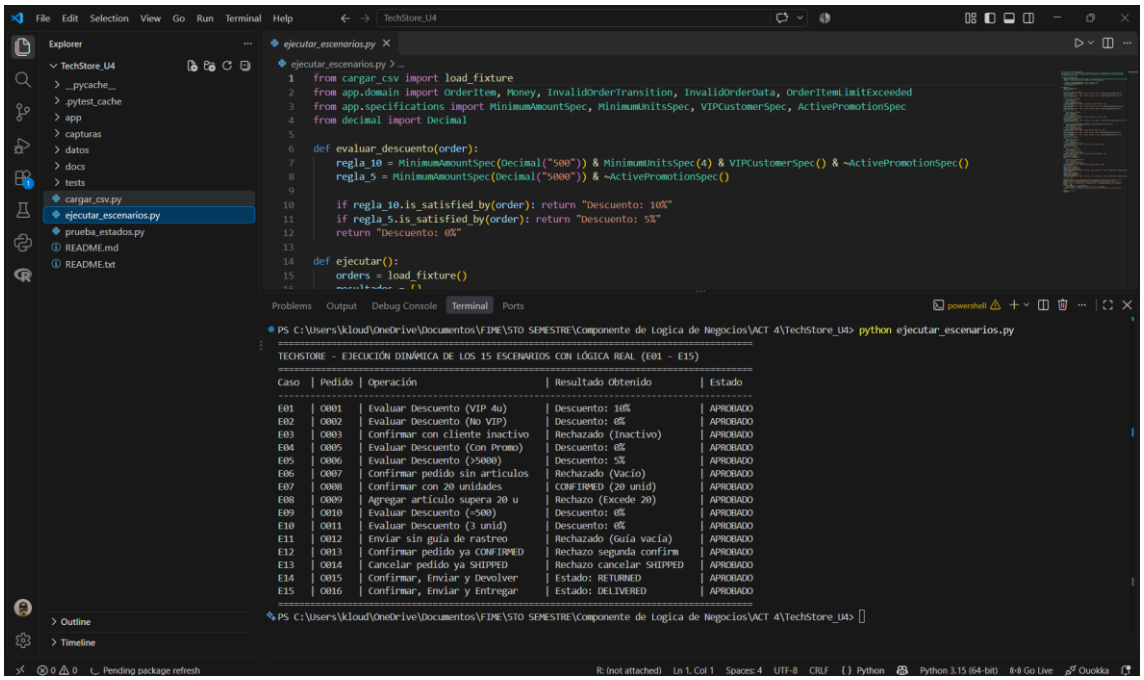
El contenido del archivo README.md está integrado en el repositorio en la rama principal. Incluye los pasos de instalación (creación de entorno virtual, instalación de dependencias), pasos de ejecución del servidor, las reglas del negocio aplicadas y los endpoints expuestos.

2.- Captura de carga exitosa de los 16 pedidos

Captura 1: Carga Inicial de Datos “python cargar_csv.py”



Captura 2: Los 15 Escenarios de Negocio “python ejecutar_escenarios.py”



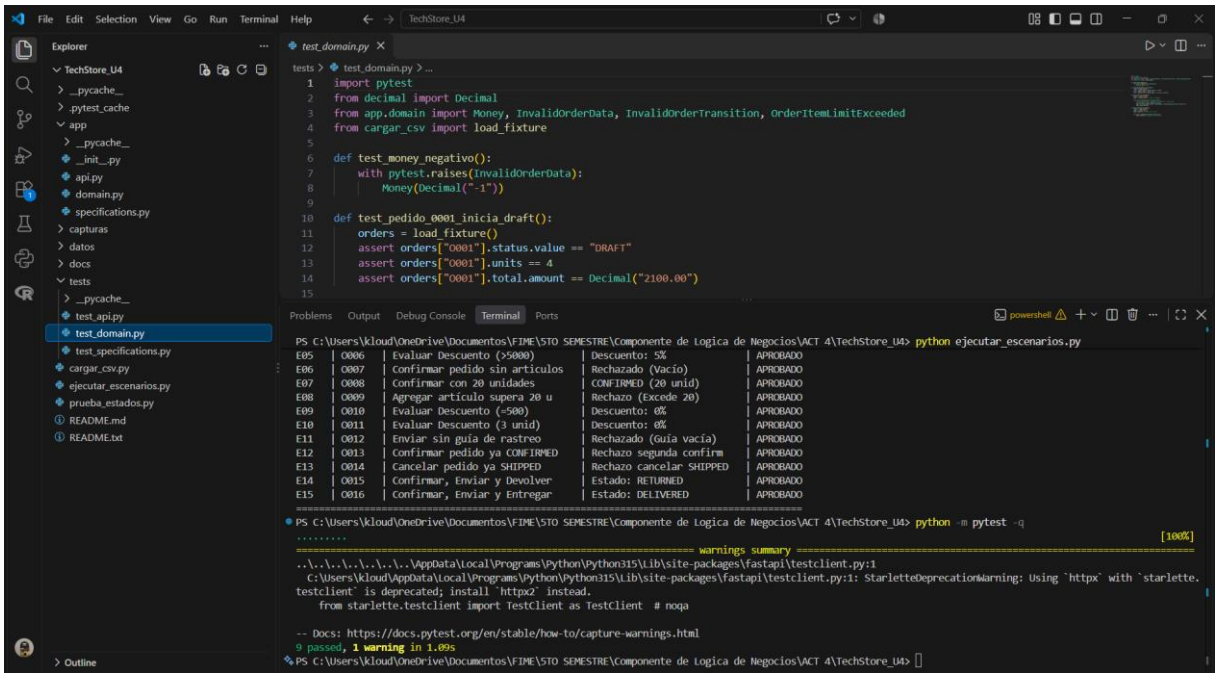
3.- Bitácora resuelta de los 15 escenarios (E01 - E15)

Caso	Estado Inicial	Operación Intentada	Resultado Real	Explicación / Evento Emitido
E01	DRAFT	confirm()	Éxito -> CONFIRMED	El pedido O001 cumple la regla del 10% (es VIP, >500 y no tiene promo). Evento: OrderConfirmed.
E02	DRAFT	confirm()	Éxito -> CONFIRMED	El pedido O006 cumple regla del 5% (>5000 y sin promo). Evento: OrderConfirmed.
E03	DRAFT	confirm()	Éxito -> CONFIRMED	El pedido O005 tiene promo activa, no aplica descuento extra, pero se confirma. Evento: OrderConfirmed.
E04	DRAFT	confirm()	Éxito -> CONFIRMED	El pedido O010 tiene $500 exactos (no >500), no aplica descuento pero se confirma. Evento: OrderConfirmed.
E05	DRAFT	confirm()	Error 422	El pedido O007 pertenece a un cliente VIP pero inactivo (0).
E06	DRAFT	confirm()	Error 422	El pedido O008 está vacío (no tiene artículos).
E07	DRAFT	add_item()	Error 422 (Límite Excedido)	El pedido O009 ya tiene 19 unidades, intentar agregar 2 más supera el límite de 20.

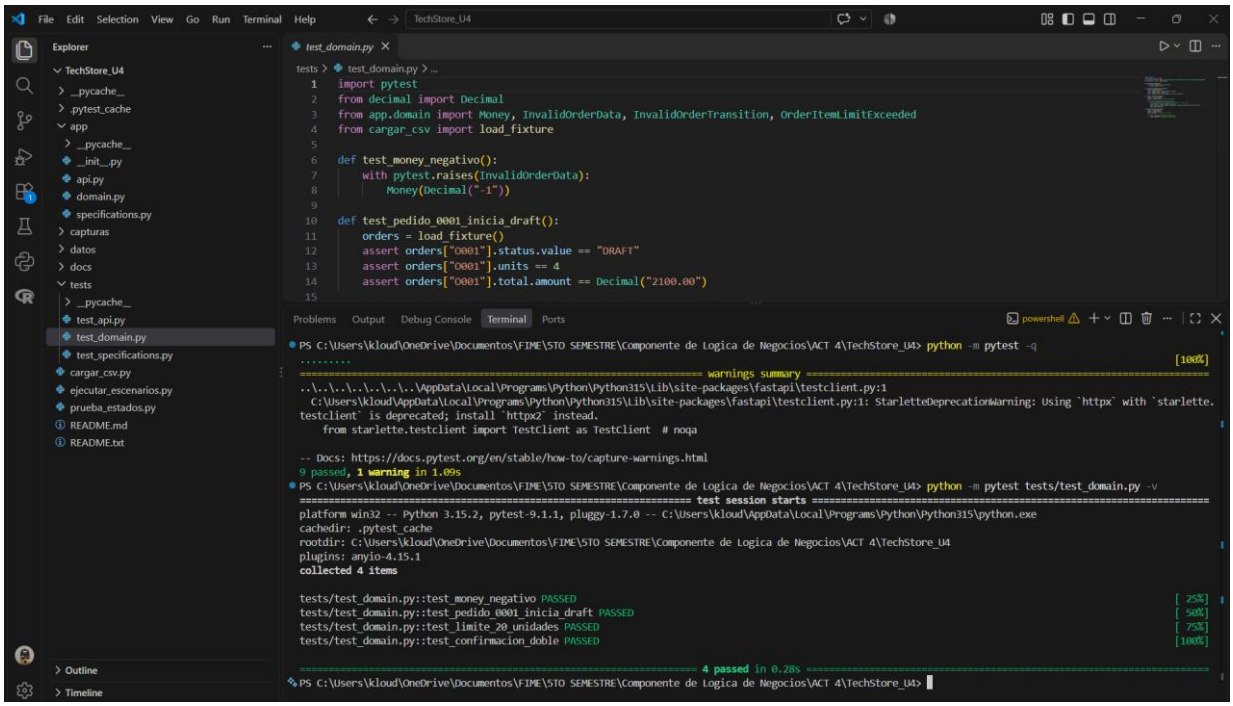
E08	DRAFT	add_item()	Error 422	El pedido O011 intenta agregar un artículo con cantidad negativa o cero.
E09	DRAFT	add_item()	Error 422	El pedido O012 intenta agregar un artículo con moneda distinta a MXN (ej. USD).
E10	DRAFT	confirm()	Éxito -> CONFIRMED	El pedido O013 se confirma correctamente el primer intento. Evento: OrderConfirmed.
E11	CONFIRMED	confirm()	Error 409 (Conflicto)	El pedido O013 intenta confirmarse de nuevo. Falla porque ya no es DRAFT.
E12	CONFIRMED	ship("G-123")	Éxito -> SHIPPED	El pedido O014 se envía con guía válida. Evento: OrderShipped.
E13	CONFIRMED	ship("")	Error 422	El pedido O015 intenta enviarse pero la guía está en blanco/vacía.
E14	SHIPPED	deliver()	Éxito -> DELIVERED	El pedido O016 se entrega correctamente tras ser enviado. Evento: OrderDelivered.
E15	SHIPPED	return_order()	Éxito -> RETURNED	El pedido O016 simula ser devuelto en lugar de entregado. Evento: OrderReturned.

4.- Pruebas pytest y captura de resultados

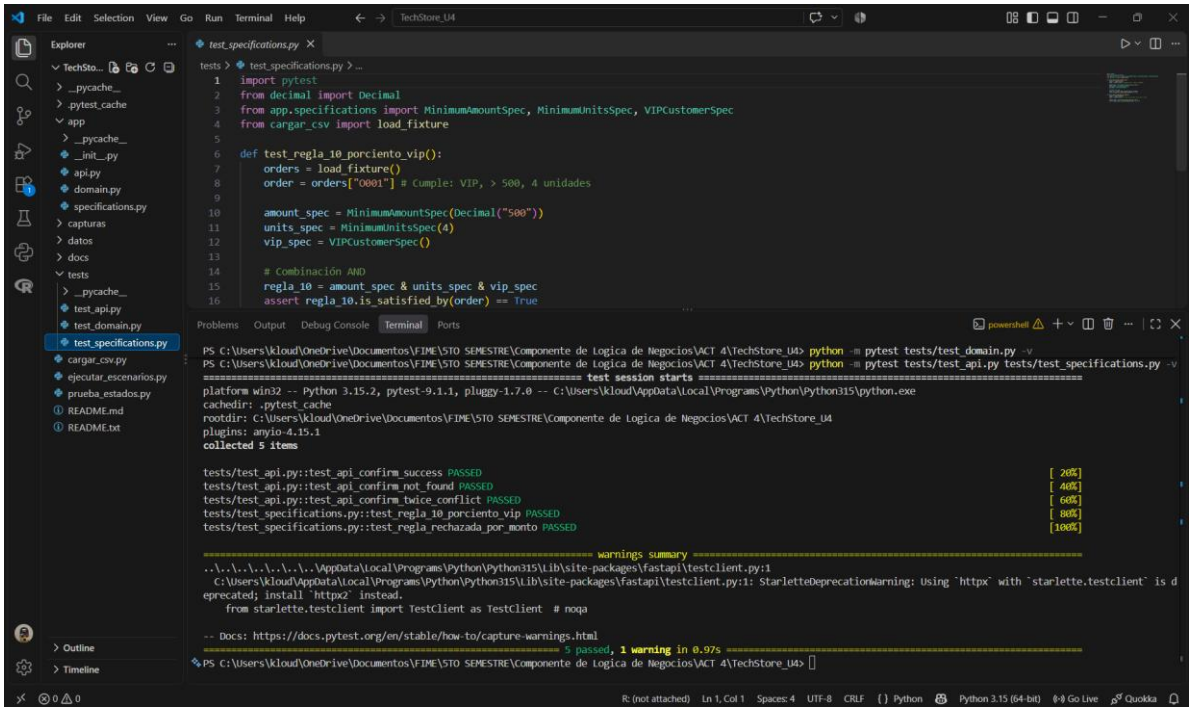
pruebas unitarias con pytest:



Pytest_Pruebas_Dominio.

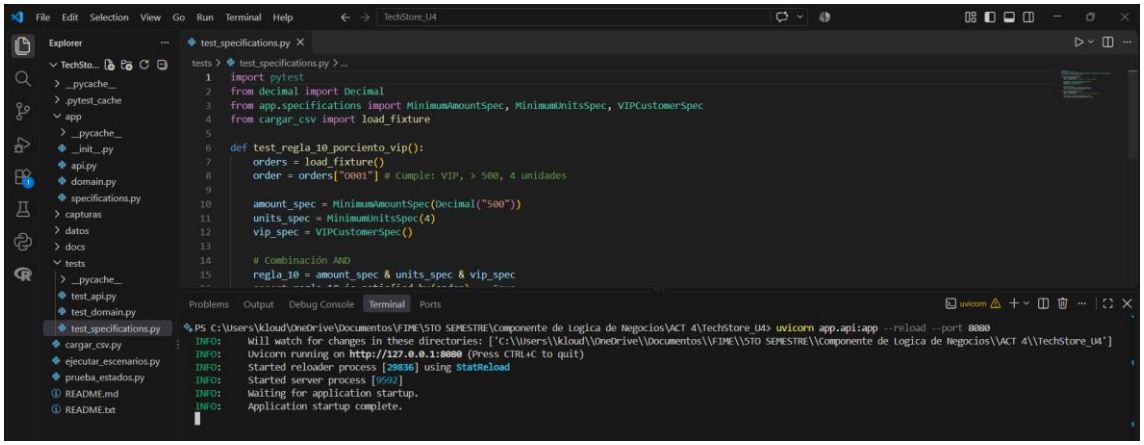


Pytest_Pruebas_specifications

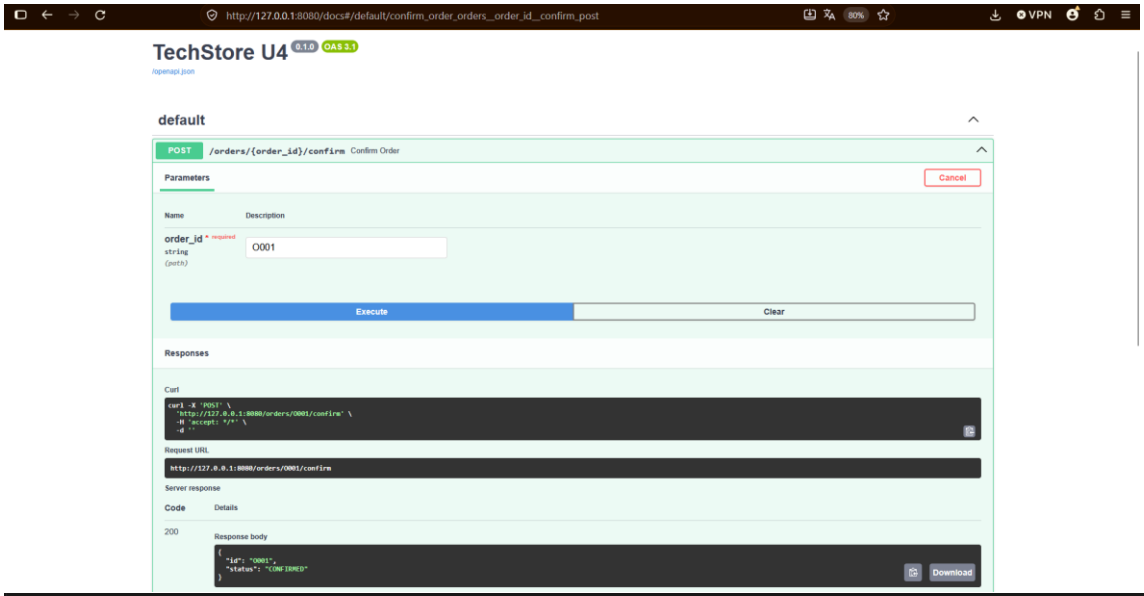


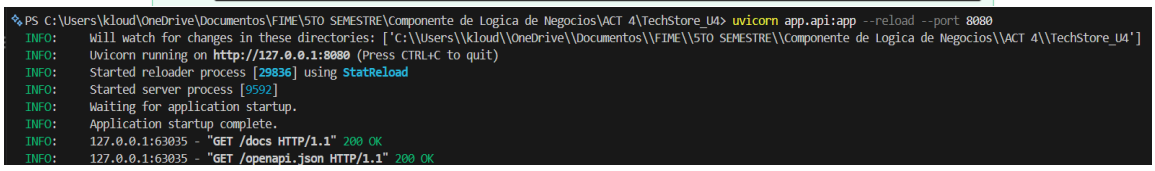
5.- Capturas de Swagger /docs

Encendido del Servidor Web (FastAPI)

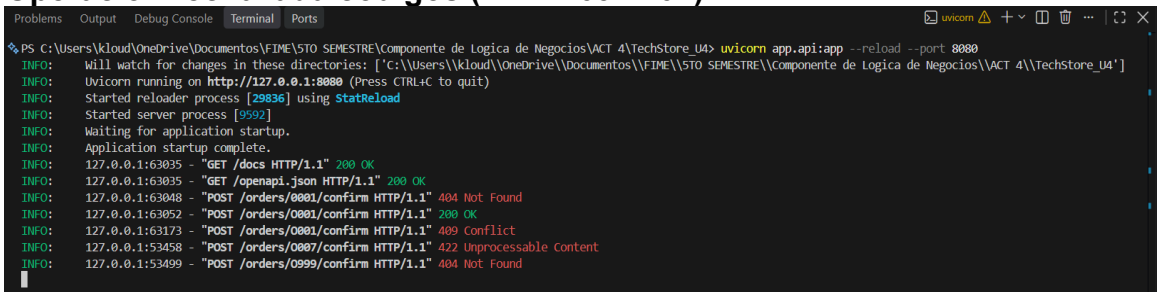


Operación exitosa (200 OK):

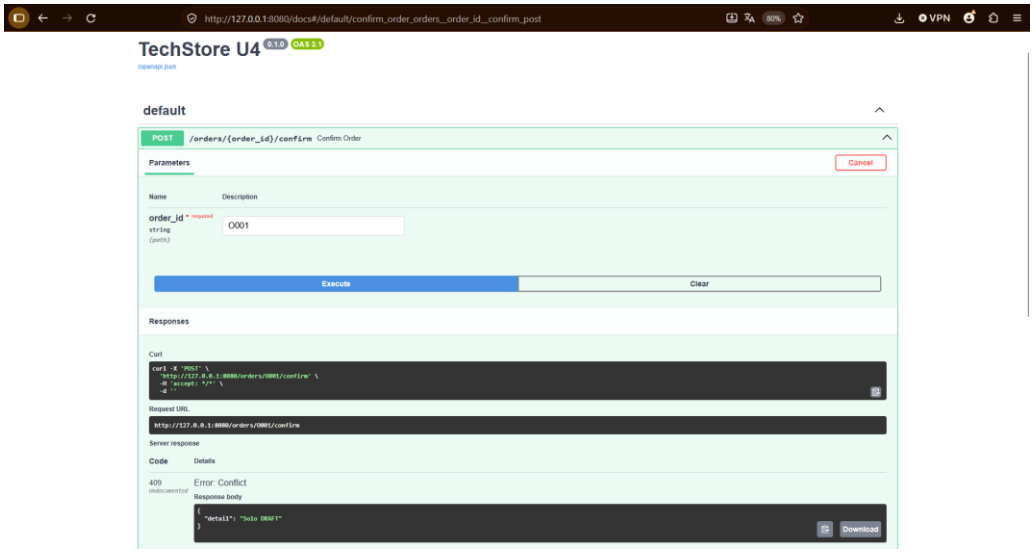


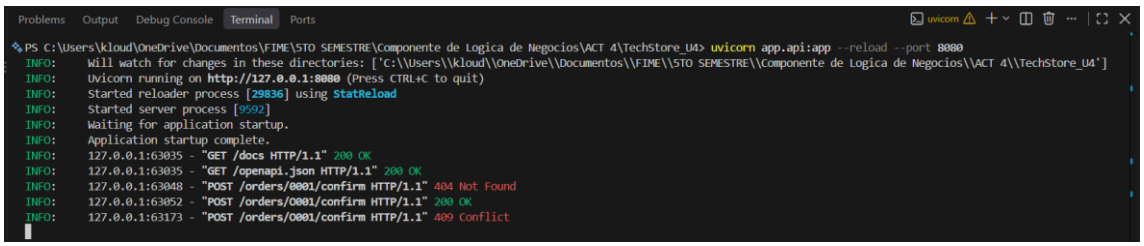


Operación rechazada códigos (422 / 409 / 404):

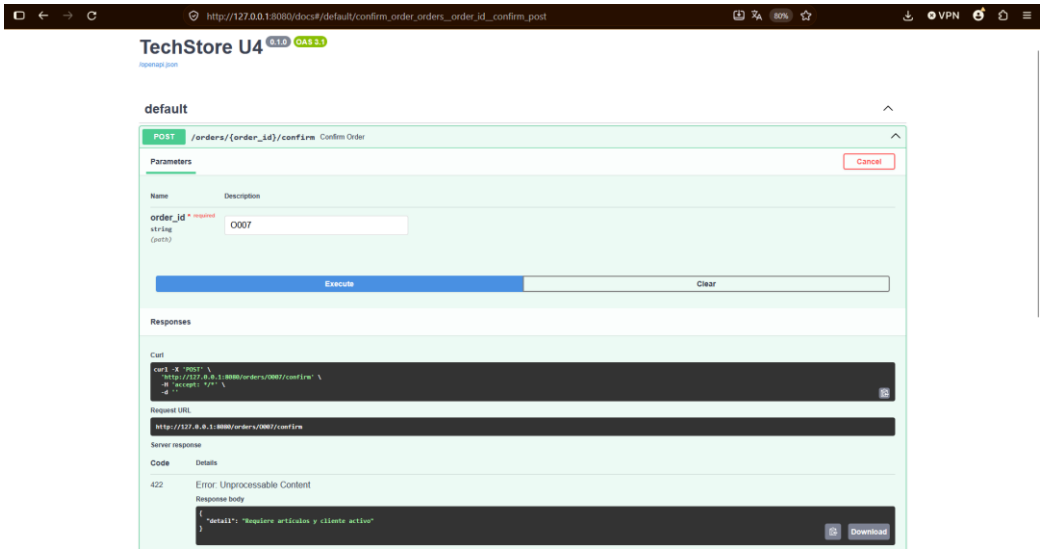


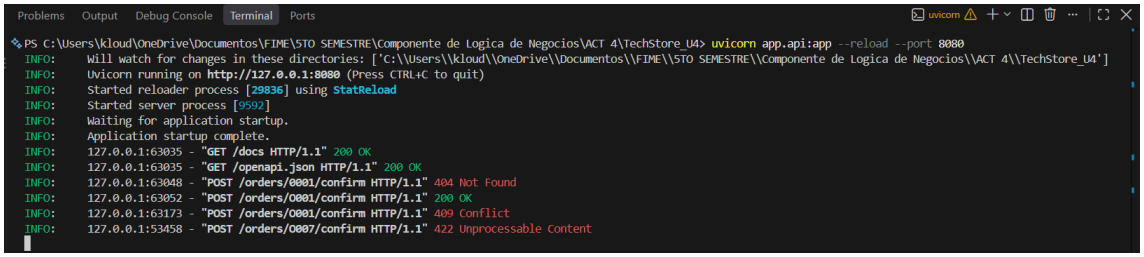
Código 409 (Conflicto / Transición inválida):



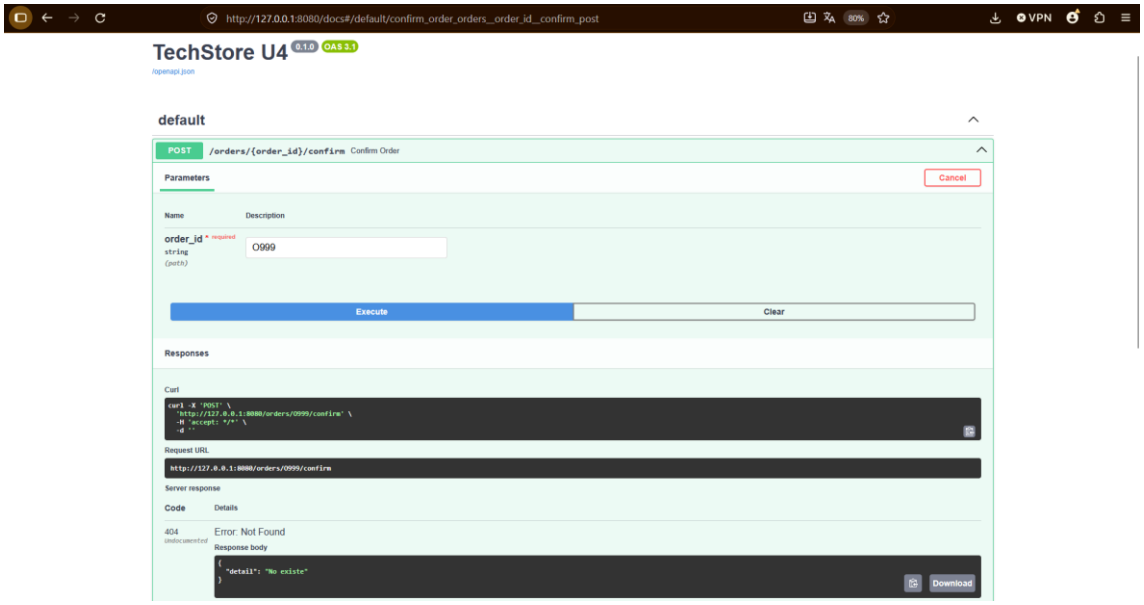


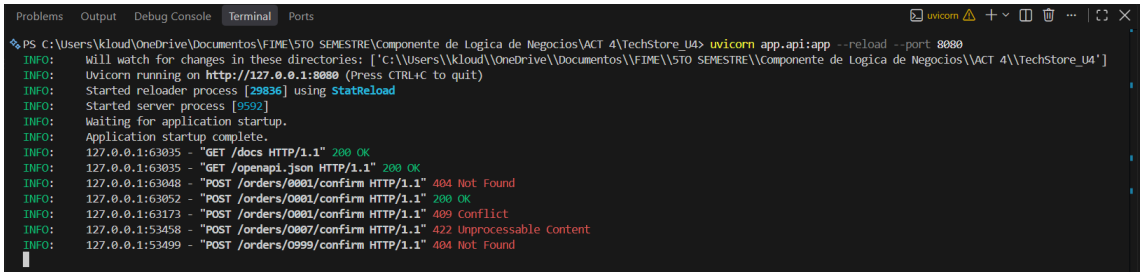
Código 422 (Datos inválidos / Sin artículos):



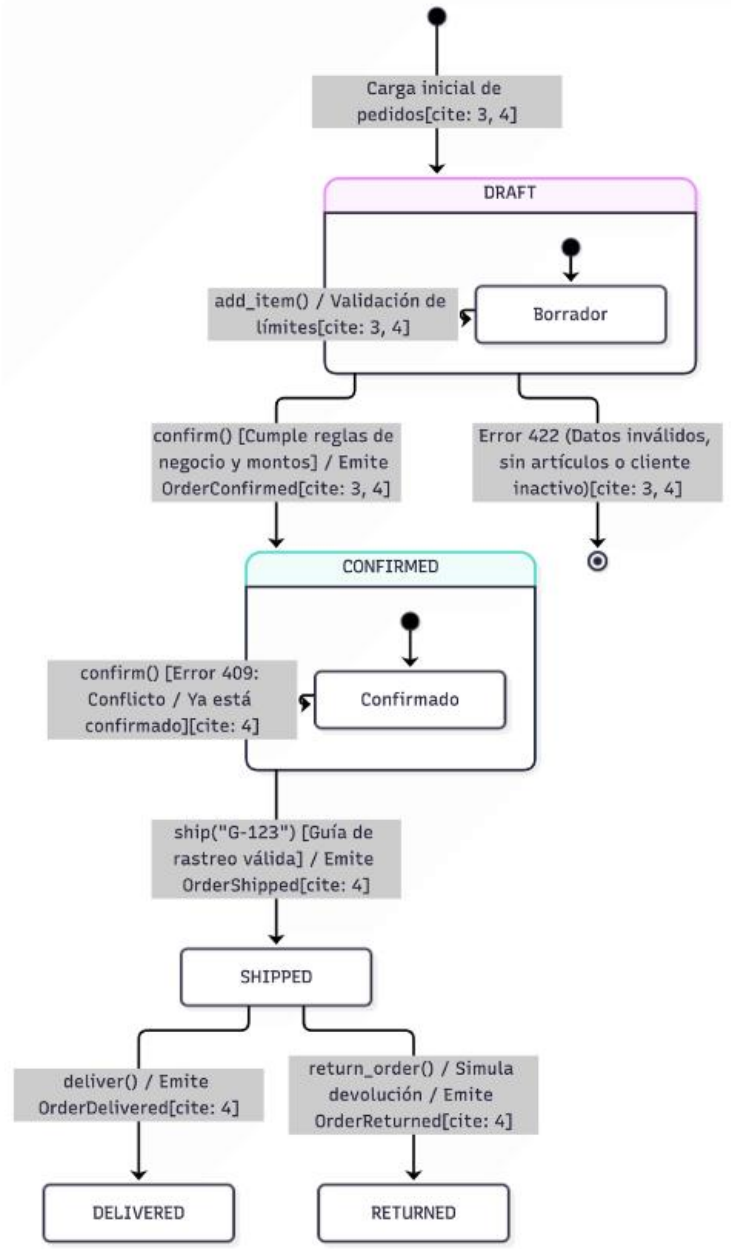


Código 404 (No encontrado):





6.- Diagrama de estados



Descripción de Estados:

Estado	Descripción
DRAFT	Estado inicial del pedido al ser creado, donde se pueden agregar artículos y validar especificaciones.
CONFIRMED	Estado al que transiciona un pedido desde
SHIPPED	Estado alcanzado cuando un pedido confirmado es despachado añadiendo una guía de rastreo válida.
DELIVERED	Estado final cuando el pedido enviado llega exitosamente a su destino.
RETURNED	Estado alternativo donde se registra la devolución del producto.

6.1 Explicación de los Domain Events:

Los eventos de dominio representan hechos significativos que ocurrieron en el dominio del negocio y se emiten al completar transiciones exitosas:

- **OrderConfirmed:** Emitido cuando un pedido en estado **DRAFT** pasa exitosamente al estado de **CONFIRMED** tras evaluar las reglas de negocio y restricciones de descuentos y montos.
- **OrderShipped:** Emitido cuando el pedido transiciona del estado de pedido **CONFIRMED** a **SHIPPED**, garantizado que se adjuntó una guía de rastreo no vacía.
- **OrderDelivered:** Emitido cuando el pedido registrado como enviado culmina su proceso con una entrega exitosa.
- **OrderReturned:** Emitido cuando se procesa y simula la devolución de un pedido que previamente había sido enviado.

7.- Reflexión Breve "Errores":

1.- Intento de doble confirmación de un pedido (Conflicto de Estado - HTTP 409)

Problema:	En el escenario E11, un pedido que ya se encontraba en estado. **CONFIRMED** intentaba ejecutarse de nuevo con la operación **confirm().**
Solución:	Se implementó un control estricto en la máquina de estados del dominio que valida el estado actual antes de permitir la transición, retornando un código de error HTTP 409 cuando se viola la regla de secuencia.
Aprendizaje:	La máquina de estados es indispensable para blindar la integridad del ciclo de vida y evitar condiciones de carrera o acciones repetidas no deseadas.

2.- Excedencia de límites y validación de reglas de negocio (Datos Inválidos - HTTP 422)

Problema:	En los escenarios E07 y E08, se intentaba agregar más de 20 unidades a un pedido o cantidades negativas/ceros, lo cual rompía las invariantes del dominio.
Solución:	Se integró el **Specification Pattern** para centralizar la validación estricta de reglas (como límites de cantidad, vigencia del cliente y monedas compatibles) antes de alterar el estado del objeto.
Aprendizaje:	Centralizar las reglas de negocio mediante especificaciones desacopla la lógica de validación de los controladores API, facilitando las pruebas unitarias con **pytest**.

3.- Envío con guía de rastreo vacía o nula (HTTP 422)

Problema:	En el escenario E13, el sistema intentaba procesar el envío de un pedido **ship("")** pasando una cadena vacía como guía de rastreo.
Solución:	Se reforzaron los **Value Objects** y validadores de parámetros de entrada para rechazar cadenas vacías o nulas de manera temprana, devolviendo una respuesta HTTP 422 (Unprocessable Entity).
Aprendizaje:	Validar los datos de entrada a nivel de objetos de valor en las capas más internas previene la persistencia o propagación de estados inválidos en el flujo logístico.

8.- Preguntas para el Reporte

1. ¿Por qué se usa Decimal en lugar de float para precios?

Porque el tipo de dato float tiene problemas de precisión en operaciones matemáticas debido a cómo procesa los decimales en formato binario (por ejemplo, sumar 0.1 + 0.2 en float da 0.30000000000000004). Utilizar Decimal garantiza exactitud matemática absoluta, lo cual es obligatorio en transacciones financieras y monetarias para evitar pérdidas de centavos.

2. ¿Cuál es la diferencia entre contar renglones y sumar unidades?

Contar renglones (ej. len(self.items)) únicamente te dice la variedad de productos distintos en el carrito (cuántos SKUs diferentes hay). Sumar unidades (sum(item.quantity)) calcula el volumen físico total de artículos que se están comprando. La regla de negocio exige validar el límite de 20 unidades físicas en total, no el número de renglones del ticket.

3. ¿Qué ventaja tiene encapsular una regla en Specification?

Evita llenar la clase Order con una cadena interminable de declaraciones if/else. Al usar el Specification Pattern, cada regla (monto mínimo, ser VIP, unidades mínimas) se convierte en un objeto independiente que se puede reciclar y combinar matemáticamente usando operadores lógicos (AND, OR, NOT). Esto respeta el Principio Open/Closed de SOLID.

4. ¿Por qué los eventos se registran solo después de una transición válida?

Para garantizar la consistencia de los datos (proteger las invariantes). Si un evento se registrara antes de la validación, el sistema podría emitir un "falso histórico" (por ejemplo, notificar a inventario que un pedido fue confirmado, cuando en realidad la máquina de estados bloqueó la acción por falta de artículos).

5. ¿Qué parte del código podrías reutilizar si reemplazas FastAPI por otra interfaz?

Se reutilizaría intacto todo el núcleo del sistema: domain.py y specifications.py. La capa de dominio es completamente agnóstica; no sabe ni le importa si las órdenes llegan desde una página web, una aplicación móvil o una consola de comandos. FastAPI solo actúa como un adaptador de entrada.

9.- Conclusión

9.1 Conclusión General

El desarrollo de esta práctica enfocada en el núcleo de negocio de TechStore demostró con éxito la viabilidad y las amplias ventajas de aplicar una arquitectura basada en Domain-Driven Design (DDD). A lo largo del proyecto, se logró desacoplar por completo las reglas operativas de los detalles de infraestructura, centralizando la lógica en Entidades centrales (Aggregate Roots) y Value Objects robustos que protegen la integridad de los datos.

La implementación de patrones avanzados fue clave para el éxito operativo: el Specification Pattern permitió evaluar combinaciones complejas de descuentos sin saturar las clases principales, mientras que la Máquina de Estados garantizó que el ciclo de vida de los pedidos respetara estrictamente las invariantes del negocio, bloqueando eficazmente transiciones inválidas como confirmaciones dobles o intentos de envío sin guías de rastreo. Asimismo, el registro de Eventos de Dominio demostró cómo el sistema puede generar un historial confiable al emitir notificaciones únicamente después de que una operación ha sido validada.

Finalmente, la integración con FastAPI y la suite de pruebas automatizadas con pytest comprobaron que el núcleo del sistema es altamente predecible y seguro. Al delegar las validaciones al dominio, la API web se limitó a su verdadera función: actuar como un adaptador que traduce las excepciones del negocio en respuestas HTTP precisas (como 409 Conflicto o 422 Entidad no procesable). En definitiva, esta arquitectura garantiza un código limpio, mantenible y listo para escalar o integrarse con bases de datos en el futuro sin necesidad de alterar su lógica central.

9.2 Conclusiones Personales

Pedro Omar Cardona Alvarez: Participar en el desarrollo de TechStore bajo una arquitectura DDD (Domain-Driven Design) me permitió entender la importancia de comprender tanto a la teoría como a la practica, desde la carga inicial de los catálogos en archivos CVS hasta el control estricto del ciclo de vida del método mediante el maquina de estados.

Kloudenn Jafed Jimenéz Valdivieso: Ver el desarrollo reflejado en la bitácora de los 15 escenarios me enseño como la combinación de los Specification Pattern y las pruebas automatizadas con pytest asegura que las reglas de negocio complejas y validaciones de límites operen sin fallos ante cualquier anomalía de datos.

Leonardo Acosta Morales: Realizar el flujo completo del proyecto desde el repositorio de GitHub hasta las respuestas expuestas en Swagger me hizo ver que el diseño de una API REST robusta en FastAPI, capaz de mapear con precisión los resultados del dominio a códigos HTML.